

MPS 法シミュレーションと DirectX 11 可視化 技術解説書

対象ソースコード

ファイル	内容
<code>mps.c</code>	MPS 法によるダムブレイク流体シミュレーション (C)
<code>mps_viewer.cpp</code>	計算結果を DirectX 11 でアニメーション表示するビューア (C++)
<code>build.bat</code>	両方をまとめてビルドするスクリプト
<code>mps_original_backup.c</code>	<code>mps.c</code> のオリジナル (コメント追加前)
<code>make_pdf.bat</code>	この解説書を PDF に変換し直すスクリプト (§ 6.6)
<code>pdf_tools¥</code>	PDF 変換に使う補助スクリプト一式

原典：Kazuya SHIBATA, Kohei MUROTANI, Seiichi KOSHIZUKA (2014)

目次

- 1. 全体像
- 2. MPS 法の理論
- 3. mps.c の実装解説
- 4. 出力ファイル仕様
- 5. DirectX 11 ビューアの設計
- 6. ビルドと実行
- 7. 操作リファレンス
- 8. トラブルシューティング
- 9. 発展課題

現在の設定：`mps.c` は **3 次元計算** に設定されている (§ 3.7)。2 次元に戻す方法も § 3.7 に記載。

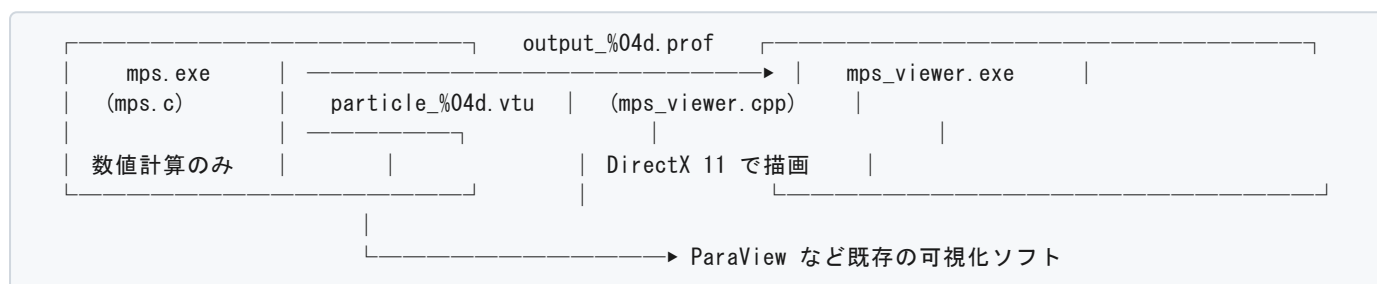
1. 全体像

1.1 何を計算しているか

ダムブレイク（ダム崩壊）問題を解いている。幅 1.0 m × 高さ 0.6 m の水槽の左端に 幅 0.25 m × 高さ 0.50 m の水柱を置き、重力で崩れて右へ流れ広がる様子を 2.0 秒間計算する。

自由表面が激しく変形し、壁に衝突して駆け上がり、しぶきが飛ぶ——格子法（有限差分法・有限体積法）が苦手とする現象であり、粒子法の有効性を示す標準的なベンチマークである。

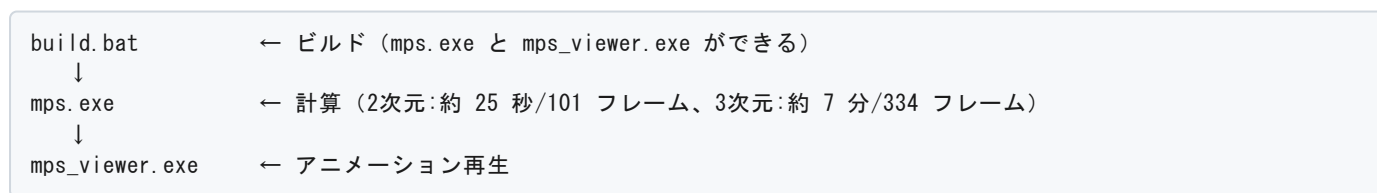
1.2 二段構えの構成



計算（ソルバ）と表示（ビューア）を分離している。理由は次の 3 点である。

- **リアルタイムでは計算できない。** 2 秒分の現象を計算するのに Release ビルドで 2 次元で約 24 秒、3 次元で約 7 分かかる（実時間の 12~240 倍。理由は § 3.6）。計算しながら滑らかに描画することはできないので、先に計算して結果を保存し、あとから再生するしかない。解像度を上げれば差はさらに広がる。
- **試行錯誤がしやすい。** 配色や視点を変えるたびに再計算する必要がない。
- **mps.c を汚さない。** 教科書のソースコードに描画コードを混ぜ込むと、アルゴリズムが読み取りにくくなる。

1.3 実行の流れ



2. MPS 法の理論

2.1 粒子法とは

流体を有限個の「粒子」の集まりとして表現し、各粒子を**ラグランジュ的に**（流れに乗って）移動させる手法。格子を使わないため、次の利点がある。

- 自由表面の大変形・分裂・合体を自然に扱える
- 移流項の計算が不要（粒子が動くこと自体が移流）
- 移流項の数値拡散が原理的に生じない

一方で、近傍粒子探索に計算コストがかかり、圧力に数値振動が生じやすいという欠点がある。

MPS 法（Moving Particle Semi-implicit method）は越塚らにより 1996 年に提案された非圧縮性流体向けの粒子法である。

2.2 支配方程式

非圧縮性ニュートン流体のナビエ・ストークス方程式と連続の式：

$$Du/Dt = -(1/\rho)\nabla P + \nu \nabla^2 u + g$$

運動方程式

$$D\rho/Dt = 0$$

連続の式（非圧縮条件）

記号	意味	本プログラムでの値
u	速度ベクトル [m/s]	Velocity[]
P	圧力 [Pa]	Pressure[]
ρ	密度 [kg/m ³]	1000.0 (FLUID_DENSITY)
ν	動粘性係数 [m ² /s]	1.0×10^{-6} (KINEMATIC_VISCOSITY)
g	重力加速度 [m/s ²]	(0, -9.8, 0)

D/Dt はラグランジュ微分。粒子法では粒子そのものが移動するので、移流項を離散化する必要がない。

2.3 重み関数（カーネル関数）

粒子 i と粒子 j の相互作用の強さを距離 r で決める関数：

$$w(r) = \begin{cases} re/r - 1 & (0 < r < re) \\ 0 & (r \geq re) \end{cases}$$

re は**影響半径**。この距離より離れた粒子とは相互作用しない。

性質と意図：

- $r \rightarrow 0$ で $w \rightarrow \infty$ に発散する。粒子どうしが極端に接近すると相互作用が強くなり、粒子の重なりを防ぐ「斥力」として働く。

- `r = re` でちょうど 0 になり、連続的に切れる。
- 対応する関数は `mps.c` の `weight()`。

影響半径の使い分け (`mps.c` 冒頭の `#define`) :

用途	値	理由
粒子数密度	$2.1 \times l_0$	自由表面判定の感度が適切になる
勾配モデル	$2.1 \times l_0$	近傍を狭くして圧力勾配を鋭く保つ
ラプラシアンモデル	$3.1 \times l_0$	広くとらないと粘性・圧力計算が不安定になる

l_0 は初期粒子間距離 (`PARTICLE_DISTANCE` = 0.025 m)。2.1 や 3.1 の「0.1」は、格子上に並んだ粒子がちょうど境界に乗って 含まれたり含まれなかったりするのを避けるための余裕である。

2.4 粒子数密度

MPS 法で最も基本的な量 :

$$n_i = \sum_{j \neq i} w(|r_j - r_i|, re)$$

これは粒子 i の周囲にどれだけ粒子が詰まっているかを表し、**流体の密度に比例する**。

- **基準粒子数密度 n^0** : 粒子が初期格子状に無限に詰まった理想状態での n の値。 `calNZeroAndLambda()` が、原点に仮想粒子を置き周囲の格子点との重みを合計して求める。
- **非圧縮条件** は「 n を n^0 に保つこと」と等価になる。
- **自由表面判定** : 表面では片側に粒子がないので n が小さくなる。 $n < \beta \cdot n^0$ ($\beta = 0.97$) なら自由表面と判定する。

対応する関数 : `calNumberDensity()`, `setBoundaryCondition()`

2.5 勾配モデル

スカラー量 ϕ の勾配を粒子間相互作用で近似する :

$$\langle \nabla \phi \rangle_i = (d / n^0) \cdot \sum_{j \neq i} [(\phi_j - \phi_i) / |r_{ij}|^2] \cdot r_{ij} \cdot w(|r_{ij}|)$$

d は空間次元数 (`DIM`)、 $r_{ij} = r_j - r_i$ 。

重要な工夫 : `mps.c` では ϕ_i の代わりに**近傍の最小圧力 $P_{\min, i}$** を使う。

$$\langle \nabla P \rangle_i = (d / n^0) \cdot \sum_j [(P_j - P_{\min, i}) / |r_{ij}|^2] \cdot r_{ij} \cdot w(|r_{ij}|)$$

こうすると $(P_j - P_{\min, i}) \geq 0$ が常に成り立つので、粒子間には**斥力しか働かない**。引力が働くと粒子が凝集して自由表面が不自然に崩れ、計算が破綻するのを防いでいる。

対応する関数 : `setMinimumPressure()`, `calPressureGradient()`

2.6 ラプラシアンモデル

$$\langle \nabla^2 \varphi \rangle_i = (2d / (\lambda \cdot n^0)) \cdot \sum_{j \neq i} (\varphi_j - \varphi_i) \cdot w(|r_{ij}|)$$

λ は補正係数で、粒子法のラプラシアン近似が拡散方程式の解析解と統計的に一致するように定める：

$$\lambda = \sum (|r_{ij}|^2 \cdot w) / \sum w$$

物理的には「重み付きの分散」であり、拡散の広がり方を合わせる役割をもつ。

対応する関数：`calNZeroAndLambda()` (λ の計算)、`calViscosity()`、`setMatrix()`

2.7 半陰的アルゴリズム

MPS 法の "Semi-implicit" (半陰的) とは、**圧力項だけを陰的に、他を陽的に**解くことを指す。1 タイムステップは 3 段階からなる。

第 1 段階：陽的計算 (仮の速度・位置)

圧力以外の力 (重力・粘性) だけで粒子を仮に動かす。

$$\begin{aligned} u^* &= u^k + (v \nabla^2 u^k + g) \cdot \Delta t \\ r^* &= r^k + u^* \cdot \Delta t \end{aligned}$$

この時点では非圧縮条件を満たしていないので、粒子数密度 n^* は n^0 からずれる。

対応：`calGravity()` → `calViscosity()` → `moveParticle()` → `collision()`

第 2 段階：陰的計算 (圧力ポアソン方程式)

ずれた粒子数密度を n^0 に戻すために必要な圧力を求める。

$$\nabla^2 P^{k+1} = (\rho / \Delta t^2) \cdot (n^* - n^0) / n^0$$

左辺にラプラシアンモデルを当てはめると、未知数 P に関する**連立一次方程式**

$$[A]\{P\} = \{b\}$$

が得られる。この行列を解くのが計算時間の大半を占める。

対応：`calPressure()` とその配下の関数群

第 3 段階：修正 (圧力勾配で速度・位置を確定)

$$\begin{aligned} u^{k+1} &= u^* - (\Delta t / \rho) \cdot \nabla P^{k+1} \\ r^{k+1} &= r^* + (u^{k+1} - u^*) \cdot \Delta t \end{aligned}$$

対応：`calPressureGradient()` → `moveParticleUsingPressureGradient()`

2.8 圧力ポアソン方程式の詳細

係数行列

ラプラスアンモデルから、内部粒子 i の行は次のようになる。

$$a = 2d / (n_{\text{lap}}^0 \cdot \lambda)$$
$$A_{ij} = -a \cdot w(r_{ij}) / \rho$$
$$A_{ii} = \sum_j (a \cdot w(r_{ij}) / \rho) + C/\Delta t^2$$

非対角項

対角項

最後の $C/\Delta t^2$ は**圧縮率項** ($\text{COMPRESSIBILITY} = 0.45 \times 10^{-9}$ [1/Pa]、水の圧縮率)。わずかな圧縮性を許すことで行列を**対角優位**にし、数値的に安定して解けるようにしている。これが無いと行列が悪条件になり、圧力が振動したり発散したりする。

右辺（ソース項）

$$b_i = \gamma \cdot (1/\Delta t^2) \cdot (n_i - n^0) / n^0$$

γ は**緩和係数** ($\text{RELAXATION_COEFFICIENT_FOR_PRESSURE} = 0.2$)。1 ステップで密度のずれを完全に打ち消そうとすると圧力が過剰に振動するため、2 割ずつ緩やかに補正する。

境界条件

粒子	扱い
自由表面粒子	$P = 0$ （大気圧基準）のディリクレ境界。方程式から外す
内部の流体粒子・壁粒子	未知数として解く
ダミー壁粒子	方程式から完全に除外（ノイマン境界 $\partial P / \partial n = 0$ の代用）

壁とダミー壁の役割

ダミー壁 (2層)

壁 (2層)

流体

この4層がないと壁粒子の
粒子数密度 n が不足し、
壁が「自由表面」と誤判定されてしまう

- 壁粒子 (WALL)：動かないが圧力は解く。流体が壁を押す反力を生む。
- ダミー壁粒子 (DUMMY_WALL)：圧力を解かない。壁粒子の n を満たすためだけに存在する。これがないと壁粒子が自由表面と誤判定され、壁が流体を押し返せなくなり、流体が壁をすり抜けてしまう。

ディリクレ境界の連結性チェック

粒子のかたまりの中に $P = 0$ を与える点（自由表面粒子）が1つも無いと、その部分の行列が**特異**になって解けない。飛沫が完全に孤立した場合などに起こりうる。

`checkBoundaryCondition()` は、自由表面粒子を出発点として「影響半径内にある粒子」を次々にたどる**幅優先探索**を行い、どの粒子が自由表面とつながっているかを調べる。つながっていない粒子については `increaseDiagonalTerm()` が対角項を 2 倍にして、無理やり解けるようにする救済処置を行う。

2.9 衝突処理

圧力計算だけでは粒子のめり込みを完全には防げないため、剛体球の衝突として速度を補正する。

距離が `COLLISION_DISTANCE` ($= 0.5 \cdot l_0$) より近く、かつ互いに近づく向きに動いている粒子対に対し、

$$\text{力積} = (1+e) \cdot (m_i m_j) / (m_i + m_j) \cdot (\text{相対速度の法線成分})$$

を与えて跳ね返す。 e は反発係数 (`COEFFICIENT_OF_RESTITUTION` $= 0.2$)。

実装上の重要点：補正後の速度は一時配列 `VelocityAfterCollision[]` に貯め、全粒子の計算が終わってからまとめて反映する。途中で `Velocity[]` を書き換えると、粒子を処理する順番によって結果が変わってしまうためである。

3. mps.c の実装解説

3.1 関数呼び出し関係

```
main
├─ initializeParticlePositionAndVelocity_for2dim  ... 初期粒子配置
│   (または _for3dim)
├─ calConstantParameter                        ... 定数の事前計算
│   └─ calNZeroAndLambda                      ... n0 と λ
│       └─ weight
└─ mainLoopOfSimulation                       ← 時間発展ループ
    ├─ calGravity                             [陽的]
    ├─ calViscosity  ───→ weight              [陽的]
    ├─ moveParticle                            [陽的]
    ├─ collision                              [陽的]
    ├─ calPressure                             [陰的]
    │   └─ calNumberDensity  ───→ weight
    │       └─ setBoundaryCondition
    │           └─ setSourceTerm
    │               └─ setMatrix  ───→ weight
    │                   └─ exceptionalProcessingForBoundaryCondition
    │                       └─ checkBoundaryCondition
    │                           └─ increaseDiagonalTerm
    │   └─ solveSimultaneousEquationsByGaussEliminationMethod
    │       └─ removeNegativePressure
    │           └─ setMinimumPressure
    └─ calPressureGradient  ───→ weight        [修正]
        └─ moveParticleUsingPressureGradient  [修正]
            └─ writeData_inVtuFormat           [出力]
                └─ writeData_inProfFormat      [出力]
```

3.2 データ構造

すべてグローバル配列で、粒子番号 **i** を添字にする。

```
static double Position[3*ARRAY_SIZE]; // Position[i*3+0] = x
                                         // Position[i*3+1] = y
                                         // Position[i*3+2] = z
static double Velocity[3*ARRAY_SIZE]; // 同様
static double Acceleration[3*ARRAY_SIZE];
static double Pressure[ARRAY_SIZE]; // スカラーなので Pressure[i]
static int ParticleType[ARRAY_SIZE];
```

ベクトル量は **i*3 + 成分** という添字規則になっている。2次元計算でも z 成分の領域は確保され、値は常に 0 である。これにより **DIM** を切り替えるだけで2次元・3次元を共通コードで扱える。

係数行列は1次元配列に2次元行列を詰めている：

```
static double CoefficientMatrix[ARRAY_SIZE * ARRAY_SIZE];
// i 行 j 列 → CoefficientMatrix[i*n + j] (n = NumberOfParticles)
```

5000×5000×8 byte = **約 200 MB** を静的領域 (.bss) に占有する。実行ファイルには含まれない (.bss は実行時に確保される) ので exe のサイズは大きくならないが、**ARRAY_SIZE** を増やすと二乗で増える点に注意。

3.3 初期粒子配置のしくみ

`initializeParticlePositionAndVelocity_for2dim()` は、格子状の候補点を走査し、その点がどの領域に属するかを **if** 文を上から順に上書き評価して決める。

```
走査範囲 (iX = -4 ... nX, iY = -4 ... nY)
↓
① ダミー壁領域か?    → DUMMY_WALL にする
② 壁領域か?          → WALL に上書き
③ 壁領域 (上部) か?  → WALL に上書き
④ 空領域か?          → 生成フラグを OFF に (水槽内部をくり抜く)
⑤ 流体領域か?        → FLUID に上書き、生成フラグ ON
↓
生成フラグが ON なら粒子を作る
```

後に書いた **if** 文が優先される構造なので、順序を入れ替えると形状が変わる。④の「空領域」で水槽の内部をくり抜くため、水槽内側に天井の壁は存在しない。そのため飛沫が水槽の上端 ($y = 0.6$ m) より上まで飛ぶことがある。

EPS ($= 0.01 \cdot l_0$) は浮動小数点の丸め誤差対策である。`x > 0.0` のような比較は誤差で判定がぶれるため、微小量を足して安定させている。

3.4 パラメーター一覧

マクロ	値	意味
<code>DIM</code>	2	空間次元数
<code>PARTICLE_DISTANCE</code>	0.025	初期粒子間距離 l_0 [m] (解像度)
<code>DT</code>	0.001	時間刻み Δt [s]
<code>FINISH_TIME</code>	2.0	計算終了時刻 [s]
<code>OUTPUT_INTERVAL</code>	20	出力間隔 ($20 \times \Delta t = 0.02$ s ごと)
<code>ARRAY_SIZE</code>	5000	粒子数の上限
<code>KINEMATIC_VISCOSITY</code>	1.0e-6	動粘性係数 ν [m ² /s]
<code>FLUID_DENSITY</code>	1000.0	密度 ρ [kg/m ³]
<code>G_X, G_Y, G_Z</code>	0, -9.8, 0	重力加速度 [m/s ²]
<code>RADIUS_FOR_NUMBER_DENSITY</code>	$2.1 \cdot l_0$	粒子数密度用の影響半径
<code>RADIUS_FOR_GRADIENT</code>	$2.1 \cdot l_0$	勾配モデル用の影響半径
<code>RADIUS_FOR_LAPLACIAN</code>	$3.1 \cdot l_0$	ラプラシアンモデル用の影響半径
<code>COLLISION_DISTANCE</code>	$0.5 \cdot l_0$	衝突判定距離
<code>THRESHOLD_RATIO_OF_NUMBER_DENSITY</code>	0.97	自由表面判定のしきい値 β
<code>COEFFICIENT_OF_RESTITUTION</code>	0.2	反発係数 e
<code>COMPRESSIBILITY</code>	0.45e-9	圧縮率 [1/Pa]
<code>RELAXATION_COEFFICIENT_FOR_PRESSURE</code>	0.2	圧力の緩和係数 γ

現在の設定について 本リポジトリの `mps.c` は現在 **3次元計算に設定されている** (§ 3.7 参照)。上の表は 2 次元用の値なので、3 次元では `DIM = 3`、`PARTICLE_DISTANCE = 0.075`、`DT = 0.003`、`OUTPUT_INTERVAL = 2` に読み替えること。2 次元に戻すには、`mps.c` 冒頭の 2 次元用 4 行の行頭の `//` を外し、3 次元用 4 行の行頭に `//` を付ける。

3.5 Visual C++ 対応で行った修正

計算式・アルゴリズムは一切変更していない。VC++ でビルド・実行できるようにするための修正のみ：

1. `#define _CRT_SECURE_NO_WARNINGS` をファイル先頭 (`#include` より前) に追加。VC++ は `sprintf` / `fopen` を「安全でない関数」として警告 C4996 を出す。`/sdl` や「警告をエラーとして扱う」設定ではビルドが失敗する。
2. `fopen()` の **NULL チェック** を追加。オリジナルはチェックが無く、書き込み禁止フォルダで実行すると NULL ポインタ参照で異常終了する。
3. `main()` の未使用引数を `(void)` キャスト。`/W4` の C4100 警告を抑止。
4. **UTF-8 BOM 付きで保存**。BOM が無いと VC++ が日本語コメントを CP932 と誤認して文字化けする。編集して保存し直すときも BOM 付き UTF-8 を維持すること。

検証結果

項目	結果
x64 <code>/W3 /O2</code> ビルド	警告 0 件
x86 <code>/W3 /O2</code> ビルド	警告 0 件
<code>/W4</code>	C4127 (<code>while(1)</code> 、 <code>if(DIM==2)</code> の定数条件) 2 件のみ。意図的な記述で無害
実行	584 粒子で正常動作
出力の一致	修正前のコードの出力とハッシュ完全一致

3.6 なぜ計算が遅いのか

ボトルネックは `solveSimultaneousEquationsByGaussEliminationMethod()` である。

- ガウスの消去法の計算量は $O(N^3)$ (N = 粒子数)
- これを毎タイムステップ (2000 回) 実行する
- $N = 584$ なら $584^3 \approx 2.0 \times 10^8$ 演算 $\times$ 2000 ステップ

実測 (既定設定・584 粒子) : `/O2` の x64 Release ビルドで **24.3 秒**。`/Od` の Debug ビルドでは **約 1.8 分**と 4~5 倍遅くなるので、必ず Release で実行すること。

さらに、近傍粒子探索がすべて **全粒子総当たり $O(N^2)$** で書かれている (`calViscosity`, `calNumberDensity`, `setMatrix`, `setMinimumPressure`, `calPressureGradient`, `collision`)。

これは教科書用に **可読性を最優先**した実装であり、実用コードでは § 9 の高速化を施す。

3.7 3 次元計算

現在 `mps.c` はこの設定になっている。

切り替え方

`mps.c` 冒頭の `#define` ブロックで、使いたいほうの 4 行を有効にする。2 次元用の行は `//` でコメントアウトしてある。

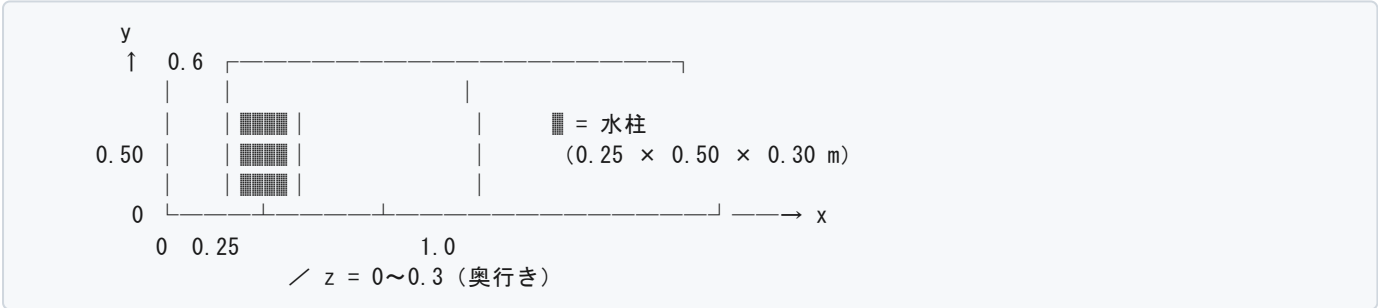
```
/* ----- 2 次元計算用（現在は無効） ----- */
// #define DIM 2
// #define PARTICLE_DISTANCE 0.025
// #define DT 0.001
// #define OUTPUT_INTERVAL 20

/* ----- 3 次元計算用（現在有効） ----- */
#define DIM 3
#define PARTICLE_DISTANCE 0.075
#define DT 0.003
#define OUTPUT_INTERVAL 2
```

`DIM` を変えるだけでなく `PARTICLE_DISTANCE` も 0.025 → **0.075 と 3 倍粗く** している。粒子数は解像度の 3 乗で効くので、3 次元で 0.025 のままにすると 粒子数が数十万個になり、 $O(N^3)$ のガウス消去法では到底終わらないからである。

計算領域

2 次元の水槽に奥行き $z = 0 \sim 0.3$ m を与えたもの。水柱は $0.25 \times 0.50 \times 0.30$ m。



2 次元との比較（実測）

項目	2 次元	3 次元
<code>PARTICLE_DISTANCE</code>	0.025 m	0.075 m
<code>DT</code>	0.001 s	0.003 s
<code>OUTPUT_INTERVAL</code>	20	2
総粒子数	584	2,680
うち流体粒子	200	72
タイムステップ数	2,000	667
出力フレーム数	101	334
計算時間（x64 Release）	24 秒	6.7 分
出力データ量	約 8 MB	約 123 MB

流体粒子がわずか 72 個 ($3 \times 6 \times 4$) しかない点に注意。2 次元は 200 個だった。粒子間距離を 3 倍粗くした代償で、3 次元のほうが総粒子数は 4.6 倍に増えているのに 流体の解像度は 2 次元よりはるかに粗い。内訳は 流体 72 / 壁 1,176 / ダミー壁 1,432 で、壁とダミー壁が全体の **97.3 %** を占める (3 次元では壁が箱状に閉じ、外側にダミー壁がもう 1 枚必要になるため)。

まともな 3 次元計算をするには §9.1 (ICCG 法) と §9.2 (バケット法) の 高速化が事実上必須である。

3 次元結果を見るとき注意

3 次元では**壁が閉じた箱**になっているため、壁粒子を表示したまま外から見ると 灰色の壁しか見えず、中の流体がまったく確認できない。

そこでビューアは、z 方向に広がりのあるデータ (= 3 次元計算) を読み込んだとき **壁を自動的に非表示**にする。

W キーでいつでも切り替えられる。 コマンドラインからは `-wall` / `-nowall` で明示指定できる。

起動時にどちらのモードで読み込んだかは、コンソールに表示される。

```
dimension      : 3D
wall display   : OFF (hidden automatically for 3D; press W to show)
```

斜めから見するには `-yaw` / `-pitch`、またはマウス左ドラッグを使う。

4. 出力ファイル仕様

4.1 .prof 形式（ビューアが読む）

`writeData_inProfFormat()` が出力するテキスト形式。

```
0.400000          ← 1行目：時刻 [s]
584              ← 2行目：粒子数
2 -0.050000 -0.050000 0.000000 0.0 0.0 0.0 0.000000 6.539419
2 -0.050000 -0.025000 0.000000 0.0 0.0 0.0 0.000000 10.29334
0 0.025000 0.025000 0.000000 1.2 -0.3 0.0 3821.44 18.92112
...              ← 3行目以降：粒子1個につき1行
```

各行の 9 項目（空白区切り）：

列	内容	単位
1	粒子種類（- 1:GHOST, 0:FLUID, 2:WALL, 3:DUMMY_WALL）	—
2-4	位置 x, y, z	m
5-7	速度 vx, vy, vz	m/s
8	圧力	Pa
9	粒子数密度	—

ファイル名は `output_%04d.prof`（`output_0000.prof` ～）。`FINISH_TIME` = 2.0 s、`OUTPUT_INTERVAL` = 20、`DT` = 0.001 なので **101 個** 生成される。

4.2 .vtu 形式（ParaView 用）

`writeData_inVtuFormat()` が出力する VTK XML 形式（UnstructuredGrid）。各粒子を「1 点だけからなるセル（`VTK_VERTEX`, type = 1）」として登録し、点データとして `ParticleType` / `Velocity`（大きさ） / `pressure` を付ける。

ParaView で `particle_*.vtu` を開けば時系列として自動認識される。本ビューアを使わずに可視化したい場合はこちらを使う。

5. DirectX 11 ビューア的设计

5.1 なぜ DirectX 11 か

候補	判断
DirectX 9	X レガシーの DirectX SDK (June 2010) の別途インストールが必要
DirectX 11	✓ Windows SDK 10 に同梱。VS をインストールすれば追加作業ゼロ
DirectX 12	X 記述量が数倍になる。この用途には過剰

DirectX 11 は `d3d11.h` / `d3dcompiler.h` / `DirectXMath.h` がすべて Windows SDK 10 に含まれており、`#pragma comment(lib, ...)` でライブラリも自動リンクできる。

5.2 プログラム構成

```
main
├─ コマンドライン解析
├─ LoadAllFrames    ... output_*.prof を全部メモリに読む
│   └─ LoadOneFrame
├─ AnalyzeFrames    ... 統計量を調べる (§ 5.7)
│   └─ PercentileOf
├─ CreateAppWindow  ... Win32 ウィンドウ生成
├─ InitD3D          ... DirectX 11 初期化
│   └─ D3D11CreateDeviceAndSwapChain
│   └─ CreateRenderTarget    ... 描画先 + Z バッファ
│   └─ CompileShaders        ... HLSL 実行時コンパイル
│   └─ CreateBuffers          ... 頂点/インスタンス/定数バッファ
├─ FitCameraToBounds ... カメラを初期配置に合わせる
├─ MainLoop          ← 毎フレーム繰り返す
│   └─ PeekMessage / DispatchMessage → WndProc (キー・マウス処理)
│   └─ UpdateAnimation    ... 経過時間でフレームを進める
│   └─ RenderFrame
│       └─ BuildInstances    ... 粒子ごとの位置と色を作る
│       └─ DrawInstanced    ... GPU で一括描画
│       └─ UpdateWindowTitle
└─ Cleanup
```

全データをメモリに載せる方式をとっている。101 フレーム × 584 粒子 × 40 byte ≈ 2.4 MB にすぎないので、毎フレームのファイル読み込みを避けたほうが再生が滑らかになる。

5.3 DirectX 11 の基本概念

オブジェクト	役割
<code>ID3D11Device</code>	リソース（バッファ、テクスチャ、シェーダ）を 生成する 係
<code>ID3D11DeviceContext</code>	描画命令を 発行する 係
<code>IDXGISwapChain</code>	描画結果を画面に出す。裏画面（バックバッファ）を管理する
<code>ID3D11RenderTargetView</code>	「ここに色を書く」という描画先の参照
<code>ID3D11DepthStencilView</code>	Z バッファ（深度バッファ）の参照

描画パイプラインの流れ：

頂点バッファ (InputLayout が並びを教える) → 入力アセンブラ (VSMain) → 頂点シェーダ (三角形をピクセルに分解) → ラスタライザ (PSMain) → ピクセルシェーダ (描画先) → 出力

本プログラムでは `D3D_DRIVER_TYPE_HARDWARE` を優先し、失敗した場合は `D3D_DRIVER_TYPE_WARP` (ソフトウェア実装) にフォールバックする。WARP は遅いが、GPU が無い環境やリモートデスクトップでも確実に描画できる。

5.4 インスタンスングによる粒子描画

584 個 (3 次元なら数万個) の粒子を 1 個ずつ `Draw()` すると、描画命令のオーバーヘッドだけで処理落ちする。そこで**インスタンスング**を使い、**1 回の命令で全粒子を描く**。

頂点バッファ (スロット0)	インスタンスバッファ (スロット1)
(-1, -1) (-1, 1) (1, -1) (1, 1) × 粒子0: 中心 (x, y, z) 色 (r, g, b, a) 粒子1: 中心 (x, y, z) 色 (r, g, b, a) 粒子2: ... : (粒子数ぶん)	
全粒子で共有 (4頂点)	粒子ごとに異なる (毎フレーム更新)

↓ `DrawInstanced(4, 粒子数, 0, 0)`

GPU が「4頂点の正方形」を粒子数ぶん描く

入力レイアウトで `D3D11_INPUT_PER_INSTANCE_DATA` を指定した要素は **頂点ごとではなくインスタンス (粒子) ごと**に進む。

```
D3D11_INPUT_ELEMENT_DESC layout[] = {
    { "POSITION", 0, DXGI_FORMAT_R32G32_FLOAT,    0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "INSTPOS",  0, DXGI_FORMAT_R32G32B32_FLOAT,  1, 0, D3D11_INPUT_PER_INSTANCE_DATA, 1 },
    { "INSTCOL",  0, DXGI_FORMAT_R32G32B32A32_FLOAT, 1, 12, D3D11_INPUT_PER_INSTANCE_DATA, 1 },
};
```

インスタンスバッファは `D3D11_USAGE_DYNAMIC` で作り、毎フレーム `Map(D3D11_MAP_WRITE_DISCARD)` で書き換える。
`WRITE_DISCARD` は「古い内容は捨ててよい」という指示で、GPU の描画完了を待たずに新しいメモリ領域をもらえるため待ち時間が生じない。

5.5 ビルボードと円形化

粒子を球として描くのは重いので、**常にカメラを向く正方形 (ビルボード)** を描き、ピクセルシェーダで円形にくり抜いて球のように見せる。

頂点シェーダ:

```
float3 worldPos = input.instPos // 粒子の中心
            + gRight.xyz * input.corner.x // カメラの右方向へ広げる
            + gUp.xyz * input.corner.y // カメラの上方向へ広げる
output.position = mul(float4(worldPos, 1.0f), gViewProj);
```

`gRight` / `gUp` にはあらかじめ**粒子半径を掛けたベクトル**を入れてあるので、`corner` (-1~+1) を掛けるだけで正しい大きさの正方形ができる。

ピクセルシェーダ:

```
float r2 = dot(input.uv, input.uv); // 中心からの距離の2乗
clip(1.0f - r2); // 1を超える=角のピクセルは描かない → 円になる

float shade = sqrt(saturate(1.0f - r2)); // 中心ほど明るく
float3 rgb = input.color.rgb * (0.40f + 0.60f * shade);
rgb *= lerp(1.0f, 0.75f, saturate((r2 - 0.55f) / 0.45f)); // 輪郭を暗く
```

`clip()` は条件が負のときにそのピクセルを破棄する HLSL の組み込み関数。 `shade` が擬似的な球の陰影を作り、輪郭を暗くすることで 隣り合う粒子の境界が見分けやすくなる。

5.6 カメラ（正射影 + オービット）

正射影（`XMMatrixOrthographicLH`）を採用している。遠近法がないので、2 次元計算の水槽が画面上で歪まず、水位や形状を目で読み取れるためである。

カメラは注視点まわりを回る **オービット方式**：

```
// ヨー角・ピッチ角から視線方向を作る
XMVECTOR forward = XMVectorSet(cos(pitch)*sin(yaw), sin(pitch), cos(pitch)*cos(yaw), 0);
XMVECTOR eye = target - forward * distance;

// 左手座標系： 右 = 上 × 前 , 上 = 前 × 右
XMVECTOR right = XMVector3Normalize(XMVector3Cross(worldUp, forward));
XMVECTOR up = XMVector3Cross(forward, right);
```

この `right` / `up` はビルボードの展開方向としてもそのまま使える。

縦横比の扱いに注意が必要だった。正射影では「画面に映る高さ」だけを指定し、幅は 高さ × 縦横比 で決まる。したがって横に長い計算領域を収めるには、必要な横幅を縦横比で割って高さに換算してから比較しなければならない。

```
float requiredHeight = vertical;
if (horizontal / aspect > requiredHeight) requiredHeight = horizontal / aspect;
g_orthoHeight = requiredHeight * 1.10f;
```

これを怠ると、横長の水槽が画面中央に小さく表示されてしまう。また `FitCameraToBounds()` は縦横比を使うため、**D3D 初期化後**に呼ぶ必要がある。

5.7 カラーマップと表示範囲の自動決定

jet カラーマップ

値を 0~1 に正規化し、青 → 水色 → 緑 → 黄 → 赤 に写す（`JetColorMap()`）。

0.00	0.25	0.50	0.75	1.00	
青	→ 水色	→ 緑	→ 黄	→ 赤	
(0, 0, 1)	(0, 1, 1)	(0, 1, 0)	(1, 1, 0)	(1, 0, 0)	

99 パーセンタイルを使う理由

MPS 法の圧力は、粒子が瞬間的に衝突した箇所だけ極端な値になる。実測では**最大 15585 Pa に対し 99 パーセンタイルは 4088 Pa**だった。最大値をスケール上限にすると、大半の粒子が下端の色（青）に潰れて何も見えない。

そこで上位 1% を外れ値として切り捨て、**99 パーセンタイルを上限**にしている。参考までに、水深 0.4 m の静水圧は $\rho gh = 1000 \times 9.8 \times 0.4 \approx 3900 \text{ Pa}$ なので、4088 Pa という上限は物理的にも妥当な値である。

実装には `std::nth_element` を使う。全体を並べ替える必要はなく、目的の位置の要素だけ確定すればよいので `std::sort` より速い。

表示範囲は「先頭フレーム」から決める

全フレームの最大範囲で合わせると、飛び散った数個のしぶき（実測で $y = 1.30 \text{ m}$ まで到達）に引っ張られて、水槽が画面のごく一部にしか表示されない。初期配置＝水槽そのものなので、**先頭フレームの境界箱**を基準にするのが最も自然である。ダミー壁は水槽の外側に広がっていて邪魔なので除外する。

粒子半径の自動推定

ビューアは `PARTICLE_DISTANCE` の値を知らないので、データから推定する。先頭フレームから最大 400 粒子を抜き出し、総当たりで**最近接粒子間距離**を求める。MPS 法では粒子が格子状に並ぶので、この値がほぼ l_0 に一致する。描画半径はその半分（粒子どうしがちょうど接して見える）。

実測：`spacing l0 : 0.02500 m` → `radius 0.01250 m`（設定値 0.025 と完全一致）。

5.8 アニメーションの時間管理

`.prof` は計算時間 `g_frameDeltaTime`（= 0.02 s）おきに出力されている。実時間 `dt` 秒が経過したら

```
進めるコマ数 = dt × 再生速度 ÷ g_frameDeltaTime
```

だけ進めれば、シミュレーションが**実時間とおりの速さ**で再生される。

```
g_currentFrame += (deltaSeconds * g_playbackSpeed) / g_frameDeltaTime;
```

経過時間は `QueryPerformanceCounter()`（高分解能タイマー）で測る。ウィンドウをドラッグしている間などに極端に大きな値になるのを防ぐため、`dt > 0.25 s` は切り捨てている。

メッセージループは `GetMessage()` ではなく `PeekMessage()` を使う。`GetMessage()` はメッセージが来るまでブロックしてしまい、入力がないとアニメーションが止まってしまうためである。

5.9 情報表示をタイトルバーにした理由

画面内に文字を描くには `DirectWrite` + `Direct2D` の相互運用が必要で、本質でないコードが数十行増える。学習用コードの見通しを優先し、フレーム番号・時刻・配色モード・再生速度は**ウィンドウのタイトルバー**に表示した。

```
MPS Viewer (DirectX 11) | Frame 30/100 t=0.600 s | Color: Pressure (x1.00) | Speed x1.00 | Playing [Loop]
```

`SetWindowTextA()` は毎フレーム呼ぶと重いので、0.1 秒に 1 回だけ更新している。

5.10 スクリーンショット機能

GPU 上のテクスチャは CPU から直接読めない。CPU が読める**ステージングテクスチャ**を作り、`CopyResource()` でコピーしてから読み出す。

```

stagingDesc.Usage          = D3D11_USAGE_STAGING;
stagingDesc.BindFlags      = 0;
stagingDesc.CPUAccessFlags = D3D11_CPU_ACCESS_READ;
g_context->CopyResource(staging, backBuffer);
g_context->Map(staging, 0, D3D11_MAP_READ, 0, &mapped);

```

保存形式は 24 bit BMP。PNG にするには WIC（Windows Imaging Component）が必要になり コードが増えるため、追加ライブラリ不要の BMP にした。BMP は行が**下から上**に並び、行サイズは 4 バイト境界に揃える必要がある点に注意。また裏画面は RGBA 順、BMP は BGR 順なので並べ替えが要る。

-shot オプションはウィンドウを非表示のまま指定フレームを描画して保存する。レポート用の静止画作成に使えるほか、開発時の動作確認にも利用した。

5.11 ビューア実装時に遭遇した問題と対処

問題	原因	対処
リンクエラー <code>__imp_TranslateMessage</code> など 17 件	<code>user32.lib</code> が自動リンクされない	<code>#pragma comment(lib, "user32.lib")</code> を追加
警告 C4566 <code>¥u301C</code>	文字列リテラル中の波ダッシュ <code>～</code> (U+301C) が CP932 で表現できない	全角チルダ <code>～</code> (U+FF5E) に置換
圧力表示が全体に青一色	一瞬の圧力スパイク (15585 Pa) がスケール上限になっていた	99 パーセントイル (4088 Pa) を上限に変更
水槽が画面中央に小さく表示	正射影の表示高さに縦横比を反映していなかった	<code>horizontal / aspect</code> と比較して高さを決定 (§ 5.6)

6. ビルドと実行

6.1 必要な環境

- Windows 10 / 11
- Visual Studio 2017 以降（「**C++ によるデスクトップ開発**」ワークロード）
- レガシーの DirectX SDK (June 2010) は**不要**

6.2 一括ビルド（推奨）

`build.bat` をダブルクリックするだけ。`vswhere.exe` で Visual Studio を自動検出するので、「開発者コマンドプロンプト」を開く必要はない。

```
build.bat
→ mps.exe           (シミュレーション)
→ mps_viewer.exe    (ビューア)
```

6.3 コマンドラインでビルド

開発者コマンドプロンプト（x64 Native Tools Command Prompt）で：

```
cl /W3 /O2 mps.c /Fe:mps.exe
cl /W3 /O2 /EHsc mps_viewer.cpp /Fe:mps_viewer.exe
```

6.4 Visual Studio の IDE でビルド

1. 「空のプロジェクト」(C++) を作成
2. 「ソースファイル」に `mps.c`（または `mps_viewer.cpp`）を追加
3. 構成を **Release / x64** に変更
4. [ビルド] → [ソリューションのビルド]

`mps.c` は拡張子が `.c` なので自動的に C コンパイラでビルドされる。デバッグ構成は最適化が無効で数倍遅くなるため、**必ず Release で実行すること**。

6.5 実行

<code>mps.exe</code>	← 2次元:約 25 秒、3次元:約 7 分。output_*.prof と particle_*.vtu を出力
<code>mps_viewer.exe</code>	← カレントフォルダの output_*.prof を再生
<code>mps_viewer.exe C:¥結果</code>	← フォルダを指定して再生

コマンドラインオプション：

オプション	意味
<code>-r <半径></code>	粒子の描画半径 [m] を手動指定（省略時は自動推定）
<code>-c <0-3></code>	起動時の配色モード（0:圧力 1:速度 2:種類 3:粒子数密度）
<code>-wall</code> / <code>-nowall</code>	壁粒子を表示する / しない（省略時は 2 次元なら表示、3 次元なら非表示）
<code>-yaw <角度></code>	起動時のカメラ水平回転角 [度]（3 次元計算用）
<code>-pitch <角度></code>	起動時のカメラ仰角 [度]（3 次元計算用）
<code>-shot <番号> <出力.bmp></code>	画面を出さずに指定フレームを BMP 保存して終了

例：

```
mps_viewer.exe -c 1           速度で色付けして起動
mps_viewer.exe . -shot 50 frame50.bmp 50コマ目を静止画で保存
mps_viewer.exe -yaw 35 -pitch 18  斜め上から見下ろす（3次元向け）
mps_viewer.exe -c 1 -yaw 35 -pitch 18 -shot 200 v200.bmp
```

6.6 この解説書を PDF に作り直す

技術解説書.md を編集したら、`make_pdf.bat` をダブルクリックすれば `技術解説書.pdf` が作り直される。

```
make_pdf.bat
[1/3] Markdown -> HTML ...
[2/3] HTML -> PDF (chrome.exe) ...
[3/3] ページ番号を付与 ...
完了: ...¥技術解説書.pdf (1,896 KB)
```

処理の中身は `pdf_tools¥` にある 3 つのスクリプトである。

ファイル	役割
<code>pdf_tools¥md2html.py</code>	Markdown を印刷用 HTML にする（体裁の CSS もここ）
<code>pdf_tools¥build_pdf.ps1</code>	3 工程をつなぐ本体。Chrome/Edge を自動で探す
<code>pdf_tools¥stamp.py</code>	ページ番号とメタデータを刷り込む

必要なもの：

- Python 3 と 3 つのパッケージ … `pip install markdown pypdf reportlab`
- Google Chrome（無ければ Microsoft Edge を使う）

体裁を変えたいときは `pdf_tools¥md2html.py` の中の `CSS` を編集する。本文書体・数式書体・余白・章ごとの改ページなどはすべてそこで指定している。

`pdf_tools¥_work.html` と `_work.pdf` は毎回上書きされる中間ファイルなので、消しても構わない。組版がおかしいときは `_work.html` をブラウザで開くと原因を追いやすい。

数式の書き方

数式は専用のフェンスで囲むと、本文とは別に **Cambria Math** で組まれる。

```
```formula
n_{i} = \sum_{j \neq i} w(|r_{j} - r_{i}|, re)
b_{i} = \gamma \cdot (1/\Delta t^2) \cdot (n_{i} - n^0) / n^0 \dots \text{ソース項}
```
```

- `_ {...}` は下付き、`^ {...}` は上付きになる（1 文字なら `n_i` のように波括弧なしでもよい）
- 行末の `...` 以降は注釈として、ゴシック体の小さな字で右側に組まれる
- 場合分けは ````cases` を使う。1 行目が左辺、2 行目以降が `値 @条件`
- 表のセルなど行中に数式を入れたいときは `<span class="mi">...</span>` で囲む

なお ````math` を使っていないのは、GitHub がそれを LaTeX として解釈してしまうためである。

罫線で描いた図はフェンスのままにしておくこと。 等幅フォントで桁を揃えて 描いてあるので、数式用の書体に変えると崩れる。

7. 操作リファレンス

ウィンドウをアクティブにして操作する。

キーボード

| キー | 動作 |
|-------|---|
| Space | 再生 / 一時停止 |
| → / ← | 1 コマ進む / 戻る（自動的に一時停止する） |
| Home | 先頭フレームへ |
| L | ループ再生の ON / OFF |
| + / - | 再生速度を上げる / 下げる（0.05～20 倍） |
| 0 | 再生速度を等倍に戻す |
| 1 | 圧力で色付け |
| 2 | 速度の大きさを色付け |
| 3 | 粒子の種類で色付け（流体=青、壁=灰、ダミー壁=暗灰） |
| 4 | 粒子数密度で色付け |
| [/] | カラースケールの上限を下げる / 上げる |
| W | 壁粒子の表示 ON / OFF（2 次元は既定 ON、3 次元は既定 OFF） |
| D | ダミー壁粒子の表示 ON / OFF（既定は非表示） |
| , / . | 粒子の描画半径を小さく / 大きく |
| F | 表示範囲を全体が入るように戻す |
| F12 | スクリーンショットを <code>shot_%03d.bmp</code> に保存 |
| Esc | 終了 |

マウス

| 操作 | 動作 |
|-------|--------------------|
| 左ドラッグ | 回転（3 次元計算の結果を見るとき） |
| 右ドラッグ | 平行移動 |
| ホイール | 拡大 / 縮小 |

使い方のコツ

- **圧力分布を見る**：1 を押し、[/] でスケールを調整する。静水圧が成立していれば、底に近いほど赤く、水面に近いほど青くなる。
- **衝突の瞬間を見る**：Space で止めて → / ← でコマ送りする。
- **壁の圧力を見る**：W で壁を表示したまま 1 を押すと、流体が壁を押す反力が色で見える。

- **自由表面の判定を確認する**：**4**（粒子数密度）にすると、表面付近で値が下がっている（青くなる）ことが確認できる。
 - **3次元の結果を見る**：起動時に壁は自動で非表示になっている。マウス左ドラッグで回すと水柱の崩れ方が立体的に分かる。**W**を押すと壁が出るが、箱が閉じているため中は見えなくなる。静止画を撮るなら **-yaw 35 - pitch 18** くらいが見やすい。
-

8. トラブルシューティング

| 症状 | 原因と対処 |
|---|--|
| no output_*.prof found | mps.exe をまだ実行していない。または別フォルダに出力されている。フォルダを引数で渡す：mps_viewer.exe <フォルダ> |
| 日本語コメントが文字化けする | ファイルが UTF-8 BOM 付きでなくなっている。エディタで「UTF-8 BOM 付き」で保存し直す |
| 警告 C4996 (sprintf / fopen) | #define _CRT_SECURE_NO_WARNINGS が #include より後ろにある。必ず先頭に置く |
| リンクエラー __imp_~ が大量に出る | #pragma comment(lib, ...) の行が欠けている。または IDE のプロジェクト設定でライブラリ指定が必要 |
| D3D11CreateDeviceAndSwapChain failed | グラフィックドライバが古い。更新する。それでも駄目ならコード内の WARP フォールバックが働くはず |
| 計算がとても遅い | Debug 構成でビルドしている。Release / x64 に変更する |
| 計算が発散する (値が NaN になる) | DT が大きすぎる。PARTICLE_DISTANCE を小さくしたら DT も小さくする |
| WARNING: There is no dirichlet boundary condition | 自由表面から孤立した粒子群が生じている。少数なら無害 (対角項 2 倍の救済処置が働く) |
| 粒子が壁をすり抜ける | ダミー壁の層数が足りていない。初期配置の領域判定を見直す |
| ビューアは動くが粒子が見えない | F キーで表示範囲をリセットする。それでも駄目なら . キーで粒子を大きくする |
| ARRAY_SIZE を増やしたらメモリ不足 | 係数行列が $\text{ARRAY_SIZE}^2 \times 8 \text{ byte}$ なので二乗で増える。10000 にすると 800 MB になる |

9. 発展課題

9.1 連立一次方程式の解法を変える（効果：大）

ガウスの消去法 $O(N^3)$ を **不完全コレスキー分解付き共役勾配法（ICCG 法）** に置き換える。係数行列は対称正定値で、かつ影響半径内の粒子としか結合しない**疎行列**なので、ICCG 法なら $O(N^{1.5})$ 程度まで落ちる。あわせて行列を疎行列形式（CRS など）で保持すれば、200 MB の配列も不要になる。

9.2 近傍粒子探索を高速化する（効果：大）

現在は全粒子総当たり $O(N^2)$ 。計算領域を影響半径程度の大きさの**格子（バケット）**に区切り、自分と隣接するバケット内の粒子だけを調べれば $O(N)$ になる。`calViscosity`, `calNumberDensity`, `setMatrix`, `setMinimumPressure`, `calPressureGradient`, `collision` のすべてに効く。

9.3 並列化

OpenMP を使うなら、粒子ループの先頭に `#pragma omp parallel for` を付けるだけで効く箇所が多い（`calViscosity`, `calNumberDensity`, `calPressureGradient` など）。ただし `collision` は一時配列に書くので安全だが、`setMatrix` は同じ対角要素に加算するため競合に注意すること。

9.4 3 次元計算の解像度を上げる

3 次元計算そのものは §3.7 のとおり既に動作している。ただし `PARTICLE_DISTANCE` = 0.075 では**流体粒子がわずかに 72 個**しかなく、自由表面の形が粒々にしか見えない。

解像度を上げたときの粒子数と計算時間の見積もり（現状のガウス消去法 $O(N^3)$ のまま）：

| <code>PARTICLE_DISTANCE</code> | 流体粒子数 | 総粒子数の目安 | 計算時間の目安 |
|--------------------------------|---------|----------|---------|
| 0.075（現在） | 72 | 2,680 | 6.7 分 |
| 0.050 | 約 240 | 約 7,600 | 数時間 |
| 0.025 | 約 1,900 | 約 55,000 | 現実的でない |

0.050 の時点で `ARRAY_SIZE` 5000 を超えるため、配列サイズの拡大も必要になる。さらに `CoefficientMatrix` は `ARRAY_SIZE2 × 8 byte` なので、`ARRAY_SIZE` = 10000 にすると 800 MB を占有してしまう。

したがって解像度を上げるには §9.1（ICCG 法 + 疎行列）と §9.2（バケット法）が事実上必須である。この 2 つを実装すれば、同じ計算時間で 1 桁細かい解像度が扱えるようになる。

9.5 ビューアの拡張

- ・ **カラーバーの表示**：現在は数値範囲がタイトルバーにしか出ない。Direct2D + DirectWrite でカラーバーと目盛りを描くと定量的に読み取れるようになる。
- ・ **速度ベクトルの矢印表示**：流れ場の向きが直感的に分かる。
- ・ **連番 BMP の自動出力**：全フレームを `-shot` で出力して動画化する。
- ・ **断面表示**：3 次元計算で、ある平面付近の粒子だけを表示する。

- **粒子の軌跡（パスライン）表示**：特定の粒子を追跡して線で結ぶ。

9.6 物理モデルの拡張

- 表面張力モデルの追加
 - 複数流体（密度の異なる流体の混合）
 - 剛体との連成（浮体シミュレーション）
 - 温度場との連成（自然対流）
-

付録：参考となる対応表

数式と関数の対応

| 数式 | 関数 |
|---|---|
| $w(r) = re/r - 1$ | <code>weight()</code> |
| n^0, λ | <code>calNZeroAndLambda()</code> |
| $n_i = \sum w(r_{ij})$ | <code>calNumberDensity()</code> |
| $v\nabla^2 u$ | <code>calViscosity()</code> |
| $u^* = u + a\Delta t, r^* = r + u^*\Delta t$ | <code>moveParticle()</code> |
| $\nabla^2 P = (\rho/\Delta t^2)(n^* - n^0)/n^0$ の右辺 | <code>setSourceTerm()</code> |
| 同 左辺（係数行列） | <code>setMatrix()</code> |
| $[A]\{P\} = \{b\}$ の求解 | <code>solveSimultaneousEquationsByGaussEliminationMethod()</code> |
| ∇P （勾配モデル） | <code>calPressureGradient()</code> |
| $u^{k+1} = u^* - (\Delta t/\rho)\nabla P$ | <code>moveParticleUsingPressureGradient()</code> |

実測値（既定設定、2次元）

| 項目 | 値 |
|----------------|--|
| 粒子数 | 584 |
| 出力フレーム数 | 101 |
| フレーム間隔 | 0.02 s（シミュレーション時間） |
| 計算時間 | 24.3 秒（x64 Release /O2） / 約 1.8 分（Debug /Od） |
| 推定粒子間距離 | 0.02500 m（設定値と一致） |
| 圧力の最大値 | 15585 Pa |
| 圧力の 99 パーセンタイル | 4088 Pa |
| 速度の最大値 | 4.74 m/s |
| 速度の 99 パーセンタイル | 2.87 m/s |
| 計算領域（先頭フレーム） | (-0.075, -0.025) ~ (1.100, 0.600) m |